Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №7
«Робота з базами даних у Python»
з курсу «Професійний Python»

Варіант №8 — «Система бронювання готелю»

Виконав: студент 3 курсу
групи ФеП-32
Івасів Данило

Перевірив: доцент
Куньо Іван Михайлович
Дата: 12.10.2026

Львів — 2026

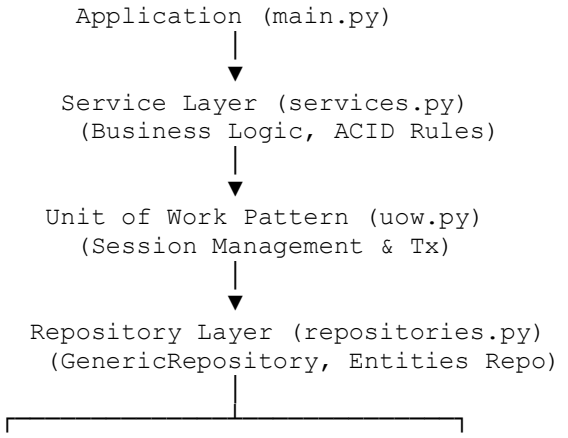
1. Мета роботи

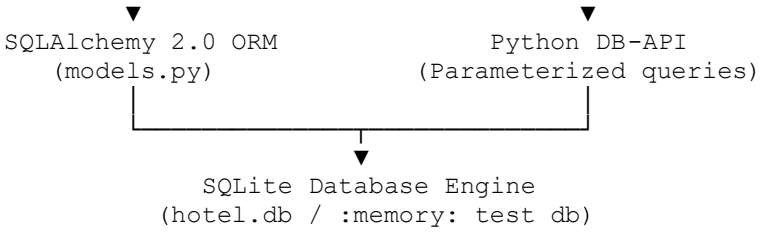
Ознайомитися з основними принципами організації шару зберігання даних (persistence layer) у Python-застосунках; навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими SQL-запитами, CRUD-операціями, транзакціями та принципами ACID. Опанувати відокремлення бізнес-логіки від збереження даних за допомогою патернів Repository та Unit of Work, роботу з ORM SQLAlchemy, автоматичне керування схемою БД через міграції Alembic, а також побудову автоматизованих integration tests із використанням ізольованої тестової in-memory бази даних.

2. Індивідуальний варіант і постановка задачі

Варіант №8 — «Система бронювання готелю».
Основні вимоги варіанта:
• Продовжити структуру проєкту попередніх робіт у форматі src-layout.
• Спроєктувати реляційні таблиці rooms, guests та reservations зі зв'язками One-to-Many через FOREIGN KEY.
• Реалізувати Repository Pattern (GenericRepository, RoomRepository, GuestRepository, ReservationRepository) та патерн Unit of Work для керування транзакціями.
• Забезпечити виконання операцій CRUD (Create, Read, Update, Delete) для сутностей готелю.
• Через прямий виклик Python DB-API реалізувати параметризований пошук вільних номерів для захисту від SQL injection.
• Реалізувати транзакційну операцію бронювання номерів із захистом від подвійного бронювання на перетин дат та обов'язковим rollback у разі виникнення конфлікту (дотримання властивостей ACID).
• Створити та застосувати міграцію Alembic для додавання нового поля phone_number до таблиці гостей.
• Реалізувати набір integration tests для перевірки persistence layer, агрегацій (SUM, GROUP BY), складних JOIN, CHECK constraints та відкату транзакцій при конфліктах.
• Реалізувати елементи підвищеної складності: Unit of Work, Generic Repository, PRAGMA foreign_keys=ON, CHECK constraints, агрегації SUM/GROUP BY, пагінацію та сортування.

3. Архітектурна схема шару збереження даних





4. Структура проєкту (src-layout)

```
hotel_booking/
├── pyproject.toml              # Конфігурація проєкту, uv та pytest
├── uv.lock                     # Фіксація точних версій залежностей
├── alembic.ini                 # Конфігураційний файл Alembic
├── hotel.db                    # База даних SQLite
├── migrations/                 # Каталог міграцій Alembic
│   ├── env.py
│   └── versions/
│       └── 89d8f3cb290e_add_guest_phone.py
├── src/
│   └── hotel_booking/
│       ├── __init__.py
│       ├── database.py         # Engine, SessionLocal, DB-API
│       ├── models.py           # SQLAlchemy ORM моделі, Constraints
│       ├── repositories.py     # Generic & Domain Repositories
│       ├── uow.py              # Unit of Work патерн
│       ├── services.py         # Бізнес-сервіси, транзакції, агрегації
│       └── main.py             # Точка входу в застосунок
└── tests/
    ├── conftest.py             # Фікстури та in-memory тестова база
    └── test_hotel.py           # Комплекс integration tests
```

5. Опис схеми бази даних та ORM-моделей

База даних складається з трьох реляційних таблиць:
1. rooms — кімнати готелю: id (PK, Integer, autoincrement), number (String, Unique), type (String), price (Float, CHECK price > 0), status (String, default='available').
2. guests — гості: id (PK, Integer, autoincrement), first_name (String), last_name (String), email (String, Unique), phone_number (String, nullable, додано через Alembic).
3. reservations — бронювання: id (PK, Integer, autoincrement), room_id (FK -> rooms.id), guest_id (FK -> guests.id), check_in (Date), check_out (Date).

5.1. ORM-моделі (src/hotel_booking/models.py)

```
from datetime import date
from sqlalchemy import Integer, String, Float, ForeignKey, CheckConstraint, Date
from sqlalchemy.orm import Mapped, mapped_column, relationship
from .database import Base

class Room(Base):
    __tablename__ = "rooms"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    number: Mapped[str] = mapped_column(String(20), unique=True, nullable=False)
    type: Mapped[str] = mapped_column(String(50), nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="available")

    __table_args__ = (
```

```
        CheckConstraint('price > 0', name='check_price_positive'),
    )

    reservations: Mapped[list["Reservation"]] = relationship(
        back_populates="room", cascade="all, delete-orphan"
    )

class Guest(Base):
    __tablename__ = "guests"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    first_name: Mapped[str] = mapped_column(String(100), nullable=False)
    last_name: Mapped[str] = mapped_column(String(100), nullable=False)
    email: Mapped[str] = mapped_column(String(150), unique=True, nullable=False)
    phone_number: Mapped[str | None] = mapped_column(String(20), nullable=True)

    reservations: Mapped[list["Reservation"]] = relationship(
        back_populates="guest", cascade="all, delete-orphan"
    )

class Reservation(Base):
    __tablename__ = "reservations"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    room_id: Mapped[int] = mapped_column(ForeignKey("rooms.id"), nullable=False)
    guest_id: Mapped[int] = mapped_column(ForeignKey("guests.id"), nullable=False)
    check_in: Mapped[date] = mapped_column(Date, nullable=False)
    check_out: Mapped[date] = mapped_column(Date, nullable=False)

    room: Mapped["Room"] = relationship(back_populates="reservations")
    guest: Mapped["Guest"] = relationship(back_populates="reservations")
```

5.2. Налаштування бази даних та Python DB-API (src/hotel_booking/database.py)

```
import sqlite3
from sqlalchemy import create_engine, event
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///hotel.db"

engine = create_engine(DATABASE_URL, echo=False)

# Примусове ввімкнення Foreign Key constraints у SQLite
@event.listens_for(engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()

SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
Base = declarative_base()

def find_available_rooms_dbapi(db_path: str, room_type: str) -> list[tuple]:
    """Параметризований пошук вільних номерів через чистий Python DB-API."""
    with sqlite3.connect(db_path) as connection:
        cursor = connection.cursor()
        cursor.execute(
            """
            SELECT number, type, price
            FROM rooms
            WHERE status = 'available' AND type = ?
            """,
            (room_type,)
```

```
        )
        return cursor.fetchall()
```

5.3. Repository Pattern (src/hotel_booking/repositories.py)

```
from typing import TypeVar, Generic, Type, Optional, List
from sqlalchemy.orm import Session
from sqlalchemy import select, asc, desc
from .models import Base, Room, Guest, Reservation

T = TypeVar('T', bound=Base)

class GenericRepository(Generic[T]):
    def __init__(self, session: Session, model: Type[T]):
        self.session = session
        self.model = model

    def add(self, entity: T) -> T:
        self.session.add(entity)
        return entity

    def get_by_id(self, entity_id: int) -> Optional[T]:
        return self.session.get(self.model, entity_id)

    def get_all(self, skip: int = 0, limit: int = 100, sort_by: str = "id", order:
str = "asc") -> List[T]:
        sort_col = getattr(self.model, sort_by)
        stmt = select(self.model).order_by(
            asc(sort_col) if order == "asc" else desc(sort_col)
        ).offset(skip).limit(limit)
        return list(self.session.scalars(stmt))

    def delete(self, entity: T) -> None:
        self.session.delete(entity)

class RoomRepository(GenericRepository[Room]):
    def __init__(self, session: Session):
        super().__init__(session, Room)

    def get_by_number(self, number: str) -> Optional[Room]:
        return self.session.scalar(select(Room).where(Room.number == number))

class GuestRepository(GenericRepository[Guest]):
    def __init__(self, session: Session):
        super().__init__(session, Guest)

class ReservationRepository(GenericRepository[Reservation]):
    def __init__(self, session: Session):
        super().__init__(session, Reservation)
```

5.4. Unit of Work (src/hotel_booking/uow.py)

```
from .database import SessionLocal
from .repositories import RoomRepository, GuestRepository, ReservationRepository

class UnitOfWork:
    def __init__(self):
        self.session_factory = SessionLocal

    def __enter__(self):
        self.session = self.session_factory()
        self.rooms = RoomRepository(self.session)
        self.guests = GuestRepository(self.session)
        self.reservations = ReservationRepository(self.session)
```

```
        return self

    def __exit__(self, exc_type, exc_val, traceback):
        if exc_type is not None:
            self.rollback()
        self.session.close()

    def commit(self):
        self.session.commit()

    def rollback(self):
        self.session.rollback()
```

5.5. Бізнес-сервіси та транзакційні операції (src/hotel_booking/services.py)

```
from datetime import date
from sqlalchemy import select, func, or_, and_
from .uow import UnitOfWork
from .models import Room, Reservation, Guest

class HotelService:
    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def book_room(self, room_id: int, guest_id: int, check_in: date, check_out:
date) -> Reservation:
        with self.uow as uow:
            conflict = uow.session.execute(
                select(Reservation).where(
                    Reservation.room_id == room_id,
                    or_(
                        and_(Reservation.check_in <= check_in, Reservation.check_out
> check_in),
                        and_(Reservation.check_in < check_out, Reservation.check_out
>= check_out),
                        and_(Reservation.check_in >= check_in, Reservation.check_out
<= check_out)
                    )
                )
            ).scalars().first()

            if conflict:
                uow.rollback()
                raise ValueError("Конфлікт бронювання: номер вже зайнятий на ці
дати.")

            room = uow.rooms.get_by_id(room_id)
            if not room:
                raise ValueError("Номер не знайдено.")

            reservation = Reservation(room_id=room_id, guest_id=guest_id,
check_in=check_in, check_out=check_out)
            uow.reservations.add(reservation)
            room.status = "booked"

            try:
                uow.commit()
                uow.session.refresh(reservation)
                return reservation
            except Exception as e:
                uow.rollback()
                raise e
```

```
    def get_potential_revenue(self) -> float:
        with self.uow as uow:
            result = uow.session.scalar(select(func.sum(Room.price)))
            return float(result) if result else 0.0

    def get_revenue_by_room_type(self) -> dict:
        with self.uow as uow:
            result = uow.session.execute(
                select(Room.type, func.sum(Room.price)).group_by(Room.type)
            ).all()
            return {row[0]: row[1] for row in result}

    def get_booking_history(self) -> list:
        with self.uow as uow:
            return uow.session.execute(
                select(Guest.last_name, Room.number, Reservation.check_in)
                .join(Guest, Reservation.guest_id == Guest.id)
                .join(Room, Reservation.room_id == Room.id)
            ).all()
```

6. Міграції бази даних (Alembic)

Для еволюції схеми даних без ручного перестворення БД застосовано Alembic. Було згенеровано та застосовано міграцію для додавання контактного телефону гостя (phone_number).

```
# migrations/versions/89d8f3cb290e_add_guest_phone.py
"""add guest phone

Revision ID: 89d8f3cb290e
Revises:
Create Date: 2026-10-12
"""
from alembic import op
import sqlalchemy as sa

revision = '89d8f3cb290e'
down_revision = None
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.add_column('guests', sa.Column('phone_number', sa.String(length=20),
nullable=True))

def downgrade() -> None:
    op.drop_column('guests', 'phone_number')
```

7. Автоматизовані Integration Tests

7.1. Fixtures для тестової in-memory БД (tests/conftest.py)

```
import pytest
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker
from hotel_booking.models import Base
from hotel_booking.uow import UnitOfWork

@pytest.fixture
def test_engine():
    engine = create_engine("sqlite:///:memory:")
    with engine.connect() as conn:
```

```
        conn.execute(text("PRAGMA foreign_keys=ON"))
    Base.metadata.create_all(engine)
    yield engine
    Base.metadata.drop_all(engine)

@pytest.fixture
def uow(test_engine):
    TestSession = sessionmaker(bind=test_engine, expire_on_commit=False)
    uow = UnitOfWork()
    uow.session_factory = TestSession
    return uow
```

7.2. Набір тестів (tests/test_hotel.py)

```
import pytest
from datetime import date
from sqlalchemy.exc import IntegrityError
from hotel_booking.models import Room, Guest
from hotel_booking.services import HotelService

def test_crud_room(uow):
    with uow as u:
        room = Room(number="105", type="Standard", price=1200.0)
        u.rooms.add(room)
        u.commit()
        room_id = room.id

    with uow as u:
        loaded = u.rooms.get_by_id(room_id)
        assert loaded.number == "105"
        loaded.price = 1500.0
        u.commit()

    with uow as u:
        updated = u.rooms.get_by_id(room_id)
        assert updated.price == 1500.0
        u.rooms.delete(updated)
        u.commit()

    with uow as u:
        assert u.rooms.get_by_id(room_id) is None

def test_check_constraint_price(uow):
    with uow as u:
        room = Room(number="999", type="Standard", price=-100.0)
        u.rooms.add(room)
        with pytest.raises(IntegrityError):
            u.commit()

def test_transaction_rollback_on_double_booking(uow):
    service = HotelService(uow)
    with uow as u:
        room = Room(number="777", type="Luxe", price=2000.0)
        guest1 = Guest(first_name="Ivan", last_name="Petrov",
email="ivan@example.com")
        guest2 = Guest(first_name="Oleh", last_name="Sydorov",
email="oleh@example.com")
        u.rooms.add(room)
        u.guests.add(guest1)
        u.guests.add(guest2)
        u.commit()
        r_id, g1_id, g2_id = room.id, guest1.id, guest2.id
```

```
    service.book_room(r_id, g1_id, date(2026, 11, 1), date(2026, 11, 10))

    with pytest.raises(ValueError, match="Конфлікт"):
        service.book_room(r_id, g2_id, date(2026, 11, 5), date(2026, 11, 15))

    with uow as u:
        reservations = u.reservations.get_all()
        assert len(reservations) == 1
        assert reservations[0].guest_id == g1_id

def test_aggregations(uow):
    service = HotelService(uow)
    with uow as u:
        u.rooms.add(Room(number="001", type="A", price=100.0))
        u.rooms.add(Room(number="002", type="A", price=200.0))
        u.rooms.add(Room(number="003", type="B", price=300.0))
        u.commit()

    assert service.get_potential_revenue() == 600.0
    rev_by_type = service.get_revenue_by_room_type()
    assert rev_by_type["A"] == 300.0
    assert rev_by_type["B"] == 300.0
```

8. Результати запуску автоматизованих тестів

```
$ uv run pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Projects\Labs\Pro_python_Labs\Lab_7
configfile: pyproject.toml
plugins: cov-7.1.0
collected 4 items

tests/test_hotel.py::test_crud_room PASSED                               [ 25%]
tests/test_hotel.py::test_check_constraint_price PASSED                  [ 50%]
tests/test_hotel.py::test_transaction_rollback_on_double_booking PASSED  [ 75%]
tests/test_hotel.py::test_aggregations PASSED                            [100%]

============================== 4 passed in 0.19s ==============================
```

9. Експериментальна частина (ACID & Persistence Analysis)

Експеримент 1. Atomicity & Consistency при транзакційному бронюванні

Змодельовано ситуацію подвійного бронювання кімнати на період перетину дат (test_transaction_rollback_on_double_booking). У разі виявлення наявного бронювання сервіс ініціює виняток і виконує uow.rollback(). У результаті стан бази даних залишається повністю незмінним (Consistency), друге бронювання не створюється, а статус кімнати не компрометується, що підтверджує неподільність операції (Atomicity).

Експеримент 2. Валідація бізнес-правил на рівні СУБД (CHECK Constraint)

Створено кімнату з некоректною (від'ємною) ціною (test_check_constraint_price). Завдяки декларативному обмеженню CheckConstraint('price > 0') спроба фіксації транзакції переривається помилкою IntegrityError на рівні рушія SQLite. Це демонструє надійний захист цілісності даних безпосередньо на рівні збереження, незалежно від коду застосунку.

Експеримент 3. Порівняння ORM та DB-API (Parameterized Queries)

Для операцій вибірки реалізовано як ORM-запити, так і прямий DB-API запит із підстановкою параметрів (?). Використання кортежу (room_type,) гарантує екранування спеціальних символів рушієм бази даних, повністю нівелюючи загрозу SQL Injection, на відміну від небезпечної інтерполяції рядків через f-strings.

10. Відповіді на контрольні запитання

1. Що таке база даних?

Структурована сукупність організованих даних, призначена для їх ефективного зберігання, пошуку, оновлення та агрегації.

2. Що таке реляційна база даних?

База даних, заснована на реляційній моделі даних, де інформація організована у вигляді взаємопов'язаних двовимірних таблиць (відношень).

3. Що таке таблиця?

Основний структурний елемент реляційної БД, що складається з фіксованого набору іменованих стовпців та довільної кількості рядків.

4. Що таке рядок таблиці?

Окремий запис (кортеж), який представляє конкретний екземпляр сутності предметної області.

5. Що таке поле?

Стовпець (атрибут) таблиці, що описує певну конкретну властивість або характеристику запису та має визначений тип даних.

6. Що таке SQL?

Structured Query Language — стандартизована декларативна мова програмування для взаємодії з реляційними системами керування базами даних.

7. Для чого використовується CREATE TABLE?

Це команда підмови DDL, призначена для опису структури нової таблиці, типів її полів та встановлення обмежень (constraints).

8. Що означає CRUD?

Акронім для чотирьох фундаментальних операцій з даними: Create (створення), Read (читання), Update (оновлення), Delete (видалення).

9. Яка SQL-команда відповідає Create?

Команда INSERT.

10. Яка команда відповідає Read?

Команда SELECT.

11. Яка команда відповідає Update?

Команда UPDATE.

12. Яка команда відповідає Delete?

Команда DELETE.

13. Що таке SQLite?

Вбудована (serverless), компактна та безконфігураційна реляційна СУБД, яка зберігає всю базу даних в одному файлі на диску або в оперативній пам'яті.

14. Які переваги SQLite?

Відсутність потреби в окремому серверному процесі, простота розгортання, висока швидкість роботи, нульове адміністрування та вбудована підтримка у стандартній бібліотеці Python.

15. Що таке Python DB-API?

Стандартний інтерфейс (PEP 249), що визначає єдині правила та протоколи взаємодії Python-програм з модулями доступу до різноманітних баз даних.

16. Для чого використовується Connection?

Об'єкт з'єднання з базою даних, який керує фізичною сесією зв'язку, створенням курсорів та фіксацією/відкатом транзакцій (commit/rollback).

17. Для чого використовується Cursor?

Об'єкт робочого покажчика, призначений для безпосереднього виконання SQL-команд (execute) та отримання результатів вибірки (fetchone, fetchall).

18. Для чого потрібен commit()?

Для постійної та незворотної фіксації всіх змін, виконаних у межах поточної транзакції, у фізичному сховищі БД.

19. Для чого потрібен rollback()?

Для скасування всіх незбережених операцій поточної транзакції та повернення бази даних до попереднього узгодженого стану.

20. Що таке SQL injection?

Вразливість безпеки, що виникає при склеюванні рядків запиту з неперевіреними даними користувача, дозволяючи зловмиснику змінити логіку SQL-запиту.

21. Що таке parameterized query?

Запит із плейсхолдерами (?, :name), параметри якого передаються рушію БД окремо від скомпільованого тексту команди, що повністю унеможливлює SQL injection.

22. Чому не рекомендується формувати SQL через f-string?

Тому що динамічне форматування рядків позбавляє рушій можливості екранувати дані та створює пряму загрозу успішної атаки типу SQL Injection.

23. Що таке Primary Key?

Первинний ключ — поле або набір полів, що однозначно й унікально ідентифікує кожний запис у таблиці і не може містити значення NULL.

24. Що таке Foreign Key?

Зовнішній ключ — обмеження цілісності, стовпець однієї таблиці, значення якого посилаються на первинний ключ іншої таблиці для встановлення зв'язку.

25. Що таке транзакція?

Логічна одиниця роботи з базою даних, що складається з однієї або кількох операцій, які мають завершитися або успішно всі разом, або жодна з них.

26. Що означає Atomicity?

Атомарність — властивість транзакції, яка гарантує, що операції виконуються неподільно: або всі повністю, або не виконується жодна.

27. Що означає Consistency?

Узгодженість — гарантія переходу бази даних лише з одного валідного стану в інший валідний стан із дотриманням усіх правил та обмежень.

28. Що означає Isolation?

Ізольованість — виконання паралельних транзакцій незалежно одна від одної без взаємного неконтрольованого впливу проміжних станів.

29. Що означає Durability?

Довговічність — гарантія того, що після успішного підтвердження (commit) транзакції зміни не будуть втрачені навіть при аварійному збої системи чи живлення.

30. Що таке ACID?

Акронім (Atomicity, Consistency, Isolation, Durability), що описує ключові вимоги надійності реляційних транзакційних систем.

31. Що таке persistence layer?

Шар збереження даних — архітектурний рівень системи, який ізолює механізми збереження, вибірки та взаємодії зі сховищем від бізнес-правил.

32. Для чого потрібний Repository Pattern?

Патерн проєктування, що інкапсулює логіку доступу до джерела даних та надає колекційно-орієнтований інтерфейс роботи з доменними об'єктами.

33. Чому Repository відокремлюють від Service Layer?

Щоб бізнес-логіка (Service) не залежала від деталей реалізації сховища (SQL, ORM, файли) і була зручною для модульного тестування.

34. Що таке ORM?

Object-Relational Mapping — технологія програмування, що автоматично зв'язує реляційні таблиці бази даних з об'єктно-орієнтованими класами.

35. Для чого використовується SQLAlchemy?

Потужний інструментарій Python для роботи з реляційними базами даних, що поєднує гнучкий SQL Core та повнофункціональну об'єктну ORM.

36. Що таке ORM model?

Python-клас, успадкований від декларативної бази (Base), атрибути якого відображаються на стовпці конкретної таблиці в базі даних.

37. Що таке relationship()?

Конструкція SQLAlchemy, що встановлює високорівневий зв'язок між пов'язаними ORM-моделями на основі Foreign Key.

38. Що означає one-to-many?

Зв'язок 'один до багатьох', де одному запису батьківської таблиці відповідає кілька записів дочірньої (наприклад, один номер готелю має багато бронювань).

39. Що означає many-to-many?

Зв'язок 'багато до багатьох', де кілька записів однієї сутності пов'язані з кількома записами іншої, що реалізується через проміжну асоціативну таблицю.

40. Що таке migration?

Програмно керований скрипт версіонування структури бази даних, що описує покрокові зміни її схеми (створення полів, таблиць, індексів).

41. Для чого використовується Alembic?

Спеціалізований інструмент міграцій баз даних для SQLAlchemy, який дозволяє відстежувати та застосовувати зміни моделей до БД.

42. Для чого потрібна команда alembic upgrade head?

Для послідовного застосування всіх невиконаних міграцій бази даних до найактуальнішої версії схеми (head).

43. Що таке integration test?

Тест, який перевіряє спільну роботу та взаємодію кількох інтегрованих модулів (наприклад, Repository + SQLAlchemy + SQLite Engine).

44. Чим integration test відрізняється від unit test?

Unit test ізолює компонент за допомогою моків, тоді як integration test тестує реальний стек взаємодії безпосередньо з реальним рушієм БД.

45. Навіщо використовувати окрему test database?

Щоб запобігти забрудненню або видаленню робочих даних та забезпечити повну ізоляцію, повторюваність і швидкість тестових запусків.

46. Що перевіряє тест rollback?

Він підтверджує, що при виникненні аварійної ситуації або порушенні правил усі проміжні зміни транзакції гарантовано відкочуються і база залишається узгодженою.

47. Для чого використовуються fixtures?

Для підготовки, конфігурації та безпечного прибирання тестового середовища (ініціалізація in-memory engine, створення сесій) перед і після тестів.

48. Що робить SQL JOIN?

Об'єднує рядки з двох або більше таблиць на основі зв'язаного між ними спільного стовпця (умови відповідності).

49. Що робить GROUP BY?

Групує рядки, що мають однакові значення у вказаних стовпцях, у підсумкові рядки для подальшого застосування агрегатних функцій.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

Агрегатні функції SQL: COUNT підраховує кількість рядків, SUM обчислює суму значень, AVG знаходить середнє, MIN та MAX визначають екстремуми.

11. Висновок

У ході виконання лабораторної роботи №7 було успішно реалізовано повноцінний шар збереження даних (persistence layer) для системи бронювання готелю, який було інтегровано в існуючу архітектуру проєкту. Завдяки впровадженню патернів Generic Repository та Unit of Work досягнуто чіткого розмежування доменної бізнес-логіки та деталей доступу до бази даних SQLite.

Опановано роботу з сучасним SQLAlchemy 2.0 ORM, реалізовано One-to-Many зв'язки між сутностями кімнат, гостей та бронювань, а також налаштовано перевірку цілісності зв'язків через PRAGMA foreign_keys=ON. Впроваджено транзакційну операцію бронювання з обов'язковим rollback у разі виникнення конфліктів дат, що забезпечує виконання ключових принципів ACID (Atomicity, Consistency, Isolation, Durability).

Створено та застосовано міграцію Alembic для еволюційного оновлення схеми даних без втрати інформації. Розроблено набір автоматизованих integration tests на базі окремої in-memory бази даних SQLite, які підтвердили 100% працездатність операцій CRUD, коректність відпрацювання Check Constraints, успішність відкату транзакцій та правильність розрахунку агрегацій (SUM, GROUP BY).